V5 自然口播稿 精修版

对应 24 页主讲。正文和媒体预算 850 秒，另留约 50 秒交接，合计约 15 分钟。实际时长未真人计时。

本组实践暂时搁置。2026-10-15 确认 A 陈云海、B 张程翔、C 赵金城、D 曹天卓，详见分工与衔接。页脚来源不用朗读；第 11 页短片边播放边解释，播放失败讲同页静态图。

A 陈云海｜问题、自动研究与递归改进

01 智能体在大模型产业研发中的应用 预算 15 秒

大家好。今天我们看智能体怎样参与大模型研发。AI 不只在帮助完成任务，也开始帮助研发 AI 本身。那么，这些改进能不能让下一轮研发变得更快、更好?

02 RSI：让改进能力进入下一轮 预算 25 秒

RSI 的英文是 Recursive Self-Improvement，中文叫递归自我改进。这里的递归，是说改进后的系统还能继续参与后续改进。早在 1965 年，Good 就设想过机器设计更好的机器。今天我们把这个问题落到研发上：不只看它产出了什么，还要看它是否更有能力做下一轮研究。

03 autoresearch：先把实验循环交给 AI 预算 35 秒

一个直观入口是 Karpathy 今年三月开源的 autoresearch。AI 修改训练代码，让小模型训练五分钟，再看验证指标，保留有效修改，继续尝试。原版用一张 GPU 就能开始。它把研究中的实验循环自动化了。RSI 则进一步追问：这些成果能不能让研究者自己更会研究？所以，自动研究是走向递归自改的一条路径，但完成一次实验循环，还没有回答后一个问题。

04 研发能力也能改进，意义就不止一次提效 预算 30 秒

这正是它值得重视的地方。一次提效，只让眼前任务做得更快；如果改进还能提高研发能力，就可能让下一轮更容易找到好方法。更强的 AI 帮助研发，研发再带来更强的系统，形成正反馈。它有机会产生复利式、甚至指数式的增长。不过，这需要增益持续成立、资源和评测跟得上，不能把一张反馈图当成已经发生的智能爆炸。

05 小实验能起步，前沿规模还要过资源关 预算 40 秒

但从小实验走到前沿模型，是另一道门槛。研究 Agent 需要有用的想法和工具，试验要付出推理成本，模型训练还需要相应算力。方法在小模型上有效，也要检验放大后是否仍有效。这让我们判断所谓 RSI 算法时，不能只看名字，要看实际规模、预算和可重复的提升。数据和 benchmark 本身可以有价值，但有这些产品，不等于已经实现持续自我改进。算法能省多少资源，资源又允许验证到哪一步，要放在一起看。

06 从自动实验，追到下一轮研发 预算 30 秒

现在，自动跑实验、优化训练程序，以及 Agent 修改自身代码，都已有公开实例。最近的 AIDE²也报告了研究 Agent 代码层的连续改进。接下来，我们重点看 Anthropic 怎样改模型，OpenAI 作者怎样组织评测，再用 Google 和开放研究补齐程序与 Agent。沿着这条研发链，最后再回答：哪些成果已经回到下一轮，还有哪些能力需要继续验证？

07 任务：让模型少迎合错误观点 预算 30 秒

先落到 Anthropic Fellows 的一项对齐研究：让模型少迎合用户的错误观点。这里有三个角色。Claude Opus 4.8 驱动研究 Agent，负责想方法、写实现；Qwen3.5-2B 是被训练的目标模型；评测器检查行为改善和能力副作用。先分清谁在研究、谁被改变，下面再看 Agent 怎样把一个想法变成训练和评测。

B 张程翔｜模型改进机制与结果

08 一轮研究要交付可审查、可训练、可评测的方法 预算 40 秒

从左往右看，一轮研究需要交付一个真正能运行的方法。研究 Agent 先读资料、看已有结果，提出训练假设；在看到本轮结果之前，先冻结 mini-paper，把方法、数据和训练目标写清楚。随后提交实现，检查代码是否和说明一致、是否违反实验规则。系统再运行训练，更新目标模型权重；典型预算约一张 H200、半小时。最后由独立评测器同时检查安全改善和能力退化，所以不是分数提高就够了，还要看有没有明显副作用。

09 下一次尝试，能读到之前的成功与失败 预算 40 秒

一次尝试结束以后，结果不会随着聊天结束就消失。多个研究 Agent 可以并行探索不同候选，而文献综述、方法代码、实验得分和失败经验会进入共享记录。新会话开始时，可以重新读取这些持久记录，再决定下一次往哪里试。所以这里的记忆是一套外部工程机制：经验能被后续会话复用，但研究 Agent 自己的模型权重并没有因此自动更新。真正被训练的是目标模型，而不是研究 Agent 本身。

10 先用激活引导，构造一个更少谄媚的教师 预算 50 秒

接下来我们看一条真正成功过的技术路线。第一步不是立刻训练学生，而是先构造一个表现更好的教师。模型回答问题时会形成内部表示，激活引导尝试对隐藏状态施加方向性干预；图上的 h 变成 h′只是教学写法。它不是把某道题的答案拼进提示词，而是让内部计算更少迎合用户的错误观点。引导后，目标模型形成一个临时更好的教师版本，但这项变化还没有写进模型权重。下一步的问题就是：怎样把这种临时变化保留下来?

11 再用 KL 蒸馏，把教师行为写进学生权重 预算 50 秒

下一步，是把教师带来的行为变化真正写进学生参数。教师和学生面对同一个输入，各自产生回答分布。教师是目标模型的引导版本，提供更少谄媚的训练目标；学生是等待训练的目标模型。训练利用 KL 约束，让学生学习教师的输出倾向，所以参数更新发生在学生身上，教师负责提供训练信号。训练结束后，再去掉临时激活引导，让学生正常推理并接受独立评测。短片只是机制教学示意；分布更接近不等于模型必然更安全，最终仍要检查行为改善与能力副作用。

12 改善空间：先弄清百分数的分母 预算 40 秒

这一页先定义指标。改善空间关闭比例，是训练带来的提升，占“基线到参考最优”原有差距的比例。看这个式子，分子是实际提升，分母是原本还能提高多少。用教学数值代入：基线 50，训练后 70，参考最优 100，分子是 20，分母是 50，所以补回了 40%的原有差距。这个 40%不是训练后正确率；论文还会用几何平均数汇总多个安全测试，所以后面的 23.3%也不能直接读成通过率。

13 同一案例中，训练目标比增加数据自由度更关键 预算 40 秒

现在看真实结果。这是在同一个谄媚任务、同一个目标模型下，对前 150 个已评分方法的比较。原样公共数据 SFT 是 6.0%，自由构造数据 SFT 是 2.9%；允许 SFT 加 KL 自蒸馏后达到 16.5%，进一步放开方法空间则达到 23.3%。所以这张图比较的是不同方法搜索条件：在这个案例里，训练目标和机制比单纯增加数据自由度更关键。每个条件只有一次搜索运行，这些结果也没有重跑 held-out 或 Petri，因此不能当作普遍规律。下一部分换到评测工作流。

C 赵金城｜研发流程与训练算法

14 另一项任务：把发布前评测跑清楚 预算 35 秒

OpenAI 工程师 Jeremy Lewi 公开过一项发布前评测工作流。研究员已经有评测目标，但要完成它，还需要找到正确工具、准备命令、处理执行问题，最后解释结果。作者把这些工作放进 Runme 笔记，让 Codex 读取工具说明和之前的记录，再协助推进。这里我们能顺着一次任务往下看：任务怎么提出，谁决定开始执行，结果又怎么留下来。

15 执行之前，研究员先审阅计划 预算 40 秒

屏幕上这句原文很直接：开始之前，等我审阅并批准计划。看下面这条顺序，先有目标和既有记录，Codex 再写出计划，研究员审阅后才执行。为什么这一步有用？因为命令能够跑通，不代表它回答了正确的问题。把计划先写出来，人就能检查准备跑哪些评测、是否漏掉关键条件。这个例子清楚展示了人在执行决策上的位置；它描述的是作者自己的工作流，不能替整个公司作同样的保证。

16 Runme 让下一次任务能接着做 预算 40 秒

Runme 把命令放在笔记里执行，输出也能跟着保存。请从上到下看：目标和计划说明为什么做，命令和输出记录具体发生了什么，解释与取舍交代下一步怎么选。换一个会话，Codex 就能读取这些记录，接着处理任务。它和前面的共享实验记录有一个共同点：上次怎么做、哪里失败、为什么这样决定，都能留给下一次。这里的记忆就保存在这些可读取的记录中。

17 AlphaEvolve 把程序候选交给自动评测 预算 40 秒

Google DeepMind 的 AlphaEvolve 把同样的生成、执行、筛选循环用在程序上。Gemini 先提出候选程序，执行器测正确性和性能，系统保留较好的候选，再继续生成。这里最重要的是，评价可以落到实际运行结果上：程序有没有算对，速度有没有更快。这类反馈适合检查算法改动。它优化的是程序，并不直接重训练模型权重。那么，改一个程序能给大模型研发带来什么收益？下一页看训练里的例子。

18 算子加速 23%，对应总训练时间减少 1% 预算 40 秒

这个案例报告，一个矩阵乘法 kernel 加速了 23%，对应 Gemini 总训练时间减少 1%。左边说的是一个算子，右边说的是完整训练。训练还有其他大量工作，所以一个环节快了很多，整段训练的收益仍可能比较小。23%落在这个算子上，1%才对应总训练时间。这个例子让我们看到，Agent 找到的一项程序改动，怎样实际进入大模型训练。

D 曹天卓｜开放研究与递归问题

19 Agent 策略与自身代码，也成为改进对象 预算 40 秒

前面改的是模型和训练程序。开放研究还把 Agent 自身作为改进对象。左边的 AgentEvolver 会生成任务、复用经验，再把行动的贡献用于奖励，训练 Agent 的策略。右边的 DGM 会改自己的工具和流程代码，比如增加补丁检查；运行评测后，把不同版本存进档案，供后续继续探索。这里要分清，训练策略与修改代码，是两种不同的改进。

20 这些案例改变了研发系统的不同部分 预算 40 秒

把前面的案例放在一起看，先问它改了哪个对象。Anthropic 训练目标模型，OpenAI 这个案例改进评测流程、留下研究记录，Google 优化训练程序。刚才的开放研究，则训练 Agent 策略或修改自身代码。这些结果分别体现在模型行为、任务完成情况和程序性能上。下一步就要问：这轮留下的成果，会怎样进入下一轮研究？

21 把上一轮的成果带进下一轮研发 预算 45 秒

上一轮的成果，有两种接法。第一种是把实验记录留下来，下次读取成功和失败，接着提出方法。前面的共享记录和 Runme 就是这样。第二种更进一步：DGM 改了 Agent 自身，后续还能从这些版本继续探索。图中两条返回的箭头，分别表示带回记录和带回修改后的 Agent。如果要把这个思路推广到基础模型研发，还要看它能否持续做出更强的后继模型。

22 下一轮，能不能研究得更好？ 预算 35 秒

怎样判断它下一轮真的研究得更好了？可以让原版和改进版，在同样预算下研究同一组新任务。先看找到的方法是否更好，再换任务、连续观察几轮，同时记录失败和人工帮助。这样有助于判断，它有没有只适应刚才那批题，是否提高研发效率，以及会不会退步或钻测试空子。这是我们提出的比较思路，本组没有做这项实验。

23 局部自改已经发生，下一步看持续研发 预算 25 秒

所以，回到开头的问题，我们已经看到特定任务里的局部自我改进：Agent 能帮助改模型，也能修改自身代码并继续探索。对于今天关注的大模型研发，下一步要检验的是，这些改进能否持续提高下一轮研究的能力，产出更强的基础模型。这就是我们对“距离递归自我改进还有多远”的回答：已有具体进展，持续的后继研发还需要跨轮次检验。

24 谢谢，欢迎提问 预算 5 秒

谢谢大家，欢迎提问。